Using OData with Python

for Stats 422

A strength of Python

We want to read the entire dataset from the web for a variety of reasons. OData (Open Data Protocol) is a standardized, REST-based protocol for building and consuming data services over the web. REST means "Representational State Transfer". It is a type of architecture for designing applications that communicate over the internet.

The **requests** library in Python happens to know how to interact with Socrata (among many other things). The requests library is not part of the Python standard library so you would need to install it (e.g., using conda or pip etc.) but it is a commonly used library.

OData examples

The example uses the Washington State Electric Vehicle population files. Many other government websites use OData to distribute data some examples

Los Angeles
San Francisco
Portland
Seattle
Memphis
Washington DC

The tricky part - the different cities and datasets need slight modifications in code to get the data read. But it is free and may be useful.

Using Python (R works too)

```
import requests
import pandas as pd
pd.set_option('display.max_columns', None)

url = "https://data.wa.gov/api/odata/v4/f6w7-q2d2?$format=json"

rows = []
next_url = url  # first page

params = {
    "$top"   : "100",
    "$select": "county,make,model,model_year"
}

while next_url: # will run while this is not empty
    print("Fetching:", next_url) # just a status indicator, where are we

    if next_url == url: # on the first pass
        r = requests.get(next_url, params=params) # apply the parameters on the first
    else:
        r = requests.get(next_url) # all other

    r.raise_for_status() # just in case we have an error
    data = r.json() # converts the response to a python object
    rows.extend(data["value"])  # make the list longer

    if "$top" in params:
        break

    next_url = data.get("@odata.nextLink") # next_url updated by what the website returns unt
```

```
Fetching: https://data.wa.gov/api/odata/v4/f6w7-q2d2?$format=json
```

```
df = pd.DataFrame(rows) # the finished result is converted to a pandas data frame
print(f"Loaded {len(df)} rows") # just print a new status indicator
```

```
Loaded 100 rows
```

```
df = df.drop(columns=['@odata.id'], errors='ignore')
df.head() # take a look at the top
```

```
      county model_year    make    model
0   Thurston       2024    AUDI     Q5 E
1     Yakima       2018    AUDI       A3
2       King       2015  NISSAN     LEAF
3       King       2019    AUDI   E-TRON
4  Snohomish       2022   TESLA  MODEL X
```

R could have read it too with the package RSocrata. You can "read" it too with a slight modification of the URL:

```
https://data.wa.gov/resource/f6w7-q2d2.json
```

Once read

Once the data is read, we could save it. Pandas is perfectly capable.

```
output_path = "washington_ev_data.csv"
df.to_csv(output_path, index=False)
print(f"Data saved to {output_path}")
```

```
Data saved to washington_ev_data.csv
```

Returning to R

We could have saved the data using R too. Python gives us many more options. Using the reticulate library we can access the pandas dataframe:

```
library(dplyr)
library(readr)
library(ggplot2)
library(reticulate)

wa_ev_data <- py$df
glimpse(wa_ev_data)
```

```
Rows: 100
Columns: 4
$ county     <chr> "Thurston", "Yakima", "King", "King", "Snohomish", "Snohomi~
$ model_year <chr> "2024", "2018", "2015", "2019", "2022", "2016", "2019", "20~
$ make       <chr> "AUDI", "AUDI", "NISSAN", "AUDI", "TESLA", "KIA", "NISSAN",~
$ model      <chr> "Q5 E", "A3", "LEAF", "E-TRON", "MODEL X", "SOUL", "LEAF", ~
```

and do whatever we want to it.

ggplot - one dimension

Recall, ggplot is simple and we can add layers to increase the control and complexity.

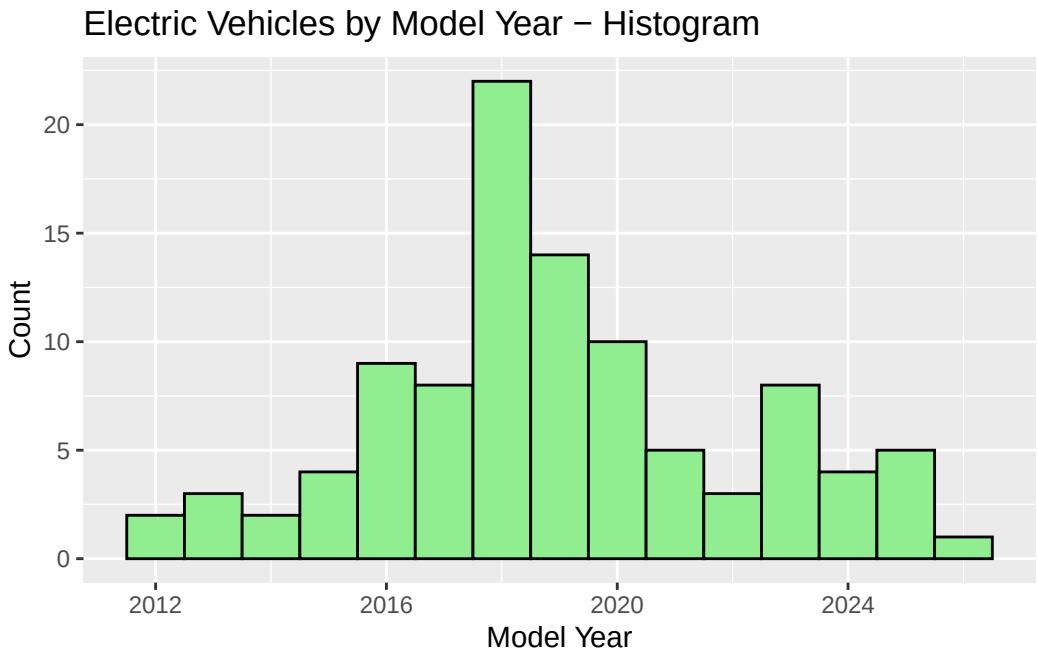
Start with the data

```
wa_ev_data <- wa_ev_data %>%
    mutate(myear = as.numeric(model_year))
```

Pass the data to ggplot. Specify your aesthetic either in the global ggplot function or in a specific geom. Choose a geom.

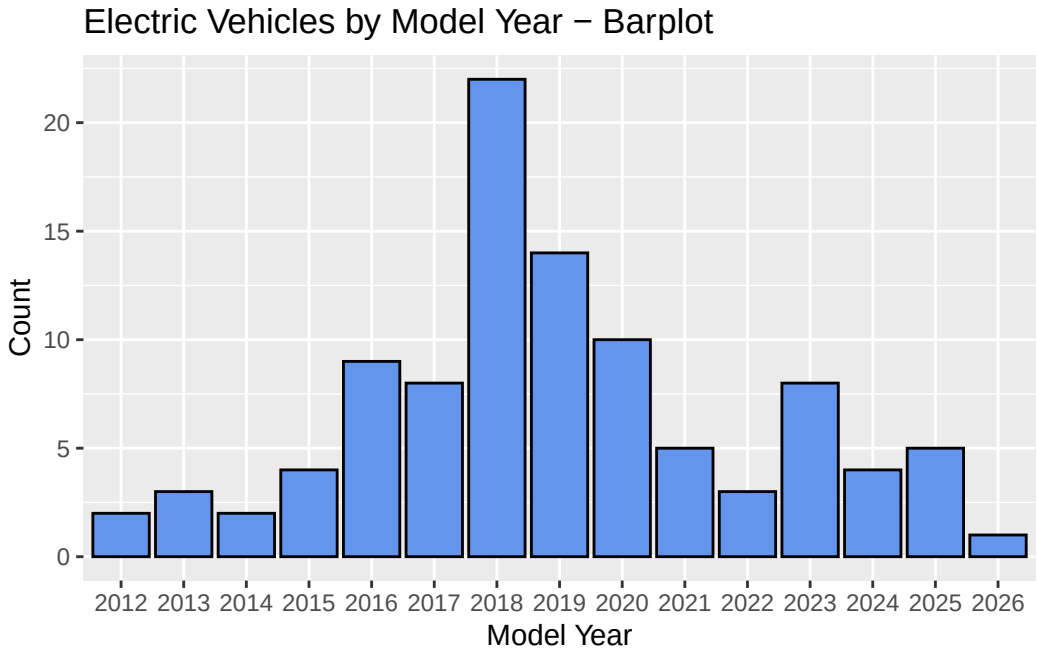
geom_histogram

```
ggplot(wa_ev_data,
    aes(x = myear)) +
  geom_histogram(
    binwidth = 1,
    fill = "lightgreen",
    color = "black") +
  labs(title = "Electric Vehicles by Model Year - Histogram",
       x = "Model Year", y = "Count")
```



geom_bar

```
ggplot(wa_ev_data) +
  geom_bar(
    aes(x = model_year),
    fill = "cornflowerblue",
    color = "black") +
  labs(title = "Electric Vehicles by Model Year - Barplot",
       x = "Model Year", y = "Count")
```



geom_col

This is for pre-aggregated data, I'll groupby/summarise the population then plot

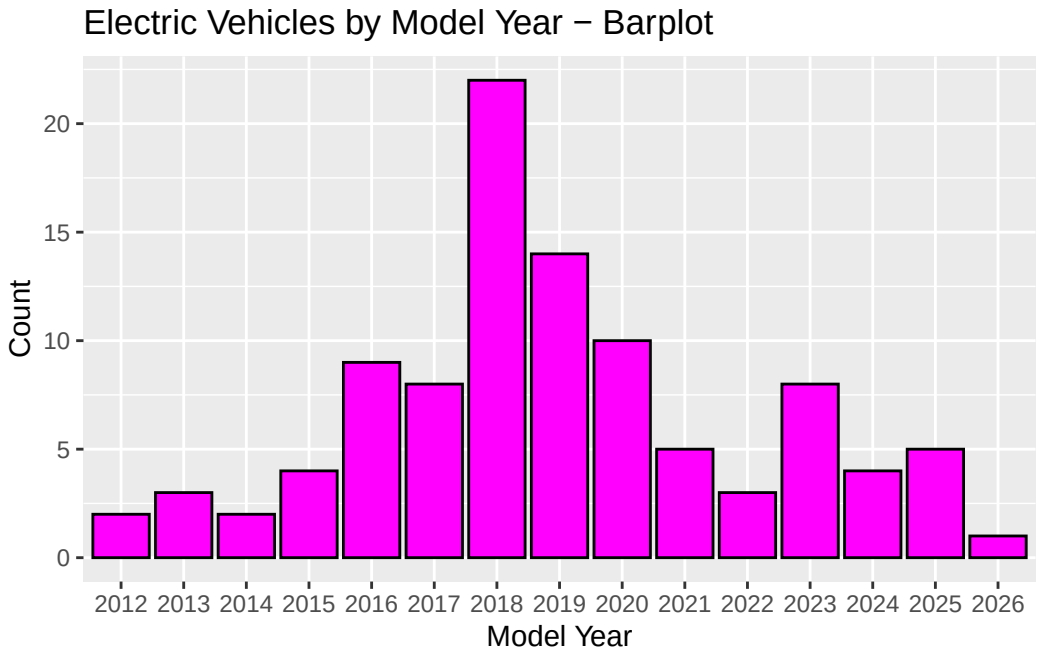
```
wa_ev_agg <- wa_ev_data %>%
  group_by(model_year) %>%
  summarise(totals = n())

wa_ev_agg
```

```
# A tibble: 15 x 2
   model_year totals
   <chr>       <int>
 1 2012            2
 2 2013            3
 3 2014            2
 4 2015            4
 5 2016            9
 6 2017            8
 7 2018           22
 8 2019           14
 9 2020           10
```

```
10  2021              5
11  2022              3
12  2023              8
13  2024              4
14  2025              5
15  2026              1
```

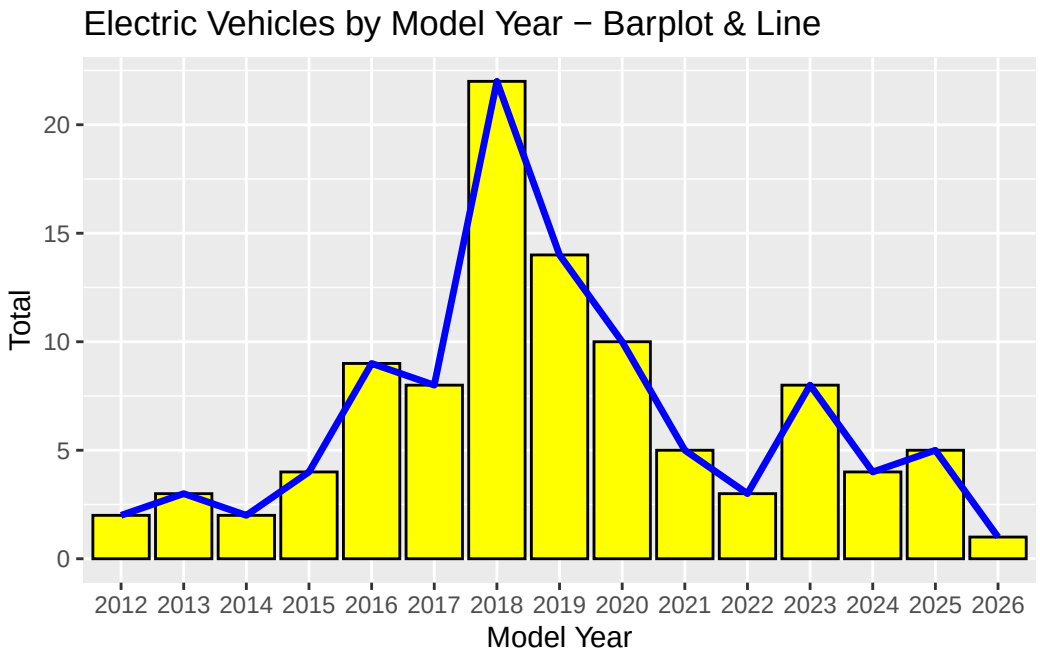
result

```
ggplot(wa_ev_agg) +
  geom_col(
    aes(x = model_year, y = totals),
    fill = "magenta",
    color = "black") +
  labs(title = "Electric Vehicles by Model Year - Barplot",
       x = "Model Year", y = "Count")
```



result 2 multiple aes()

```r
ggplot(wa_ev_agg, aes(x = model_year)) +
  geom_col(
    aes(y = totals),
    fill = "yellow",
    color = "black") +
  geom_line(
    aes(y = totals, group = 1), # forces ggplot to connect a single path
    linewidth = 1.2,
    color = "blue") +
  labs(title = "Electric Vehicles by Model Year - Barplot & Line",
       x = "Model Year", y = "Total")
```



Takeaways

- Python is very strong in the acquistion of data
- Python and R can work together
- ggplot is elegant and not as difficult as it may first seem
- but it can be slower than some of Python's visualization libraries